

Lecture 8: Time Series
Koopman Operator, Recurrent Networks
Reservoir Computing

Deep Learning in Classical and Quantum Physics

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

1

Recap of Lecture 7



JAGIELLONIAN UNIVERSITY
IN KRAKOW

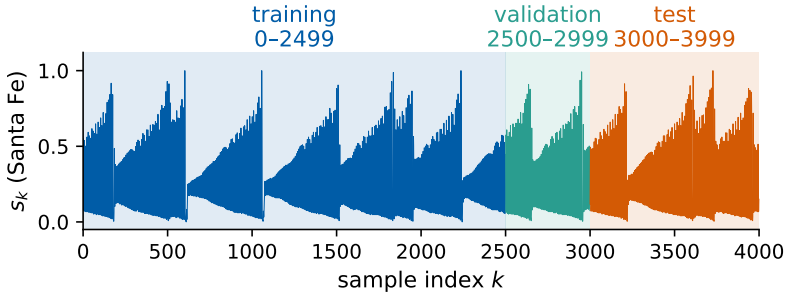
Forecasting Task

We measure one observable s_k of a hidden dynamical system and forecast its next 50 values from its past. At the present time k the model reads the window of the last L values and returns all 50 future values at once.

$$\mathbf{v}_k \coloneqq (s_{k-L+1}, \ldots, s_{k-1}, s_k), \qquad \mathbf{y}_k \coloneqq (s_{k+1}, \ldots, s_{k+50}), \qquad \hat{\mathbf{y}}_k = F(\mathbf{v}_k)$$

- the lead $\tau = 1, \ldots, 50$ counts the steps into the future, and $\hat{s}_{k+\tau}$ is the forecast for lead τ
- this is the direct forecast of Lecture 7, so the model never reads its own predictions
- every model of this lecture solves this same task on the same signals

Training, Validation and Test in Time Order



The figure shows the Santa Fe signal s_k against the sample index k, rescaled to $[0, 1]$, with the three blocks in time order.

- the models are fitted on 2387 forecast origins, tuned on 451 and tested on 951
- the test block is used once, after all choices are made, so the scores measure forecasts of an unseen future

Forecast Capacity

For each lead τ we compare the forecasts with the true values over all test origins.

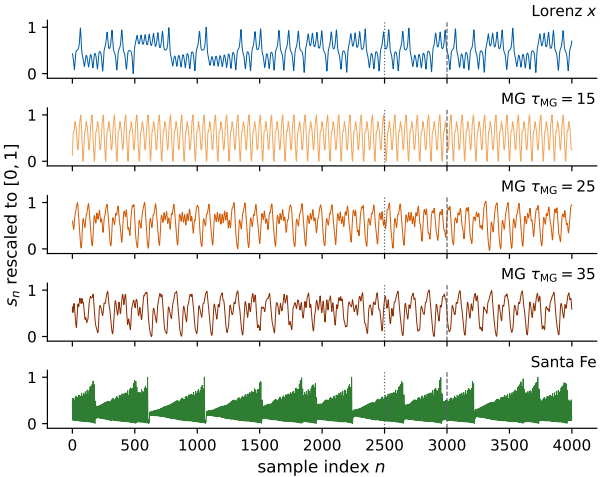
Definition (Capacity and total capacity)

$$C(\tau) := r^2(\hat{s}_{k+\tau},\, s_{k+\tau}) \in [0,1], \qquad C_{\mathrm{tot}} := \sum_{\tau=1}^{50} C(\tau) \le 50$$

- r is the Pearson correlation over the test origins, so $C(\tau) = 1$ means the forecast follows the truth up to scale and offset
- C_{tot} counts how many of the 50 leads are effectively predictable

J. Dambre *et al.*, Sci. Rep. 2, 514 (2012), doi:10.1038/srep00514

Five Signals



Lorenz (1963); Mackey, Glass (1977); Hübner *et al.* (1989)

Each panel shows the rescaled signal s_n against the sample index n. The dotted line marks the start of validation, the dashed line the start of the test block.

- Lorenz-63, the first coordinate x of the convection model
- Mackey–Glass (MG) delay equation with delay $\tau_{\mathrm{MG}} = 15$ (limit cycle) and 25, 35 (chaos)
- Santa Fe, the measured intensity of a chaotic laser

Baselines from Lecture 7

The table gives C_{tot} on the test block for the three models of Lecture 7, refitted on the windows of this lecture. Persistence repeats the last value, $\hat{s}_{k+\tau} = s_k$.

model	Lorenz	MG 15	MG 25	MG 35	Santa Fe
persistence	8.4	19.5	13.3	11.8	5.5
autoregressive (AR)	15.4	49.8	33.1	38.0	16.5
nonlinear vector AR (NVAR)	15.4	50.0	45.5	48.6	34.7

- the limit cycle is forecast almost perfectly by the linear model, and the chaotic Mackey–Glass signals gain from the products of NVAR
- on Lorenz and on the laser a large gap to the maximum of 50 remains

2

Lifting and the Koopman Operator



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Why Linear Readouts Can Work

All our models forecast with a linear combination of features, $\hat{s}_{k+\tau} = \mathbf{w}^\top \boldsymbol{\varphi}_k$, while the dynamics behind the signals is nonlinear. This section explains

- when nonlinear dynamics has an exact linear description
- which features make this possible
- why it works for periodic signals and fails for chaos

The key idea is from 1931: follow functions of the state instead of the state itself.

B. O. Koopman, PNAS 17, 315 (1931), doi:10.1073/pnas.17.5.315

A Nonlinear Map That Becomes Linear

Consider a map of two variables with one nonlinear term,

$$x_{k+1} = a\,x_k, \qquad y_{k+1} = b\,y_k + c\,x_k^2.$$

No 2×2 matrix maps (x_k, y_k) to (x_{k+1}, y_{k+1}), because of x_k^2. We add x^2 as a third variable. It evolves linearly too, $x_{k+1}^2 = a^2 x_k^2$, so

$$\mathbf{z}_{k+1} = K\mathbf{z}_k, \qquad \mathbf{z} = \begin{pmatrix} x \\ y \\ x^2 \end{pmatrix}, \qquad K = \begin{pmatrix} a & 0 & 0 \\ 0 & b & c \\ 0 & 0 & a^2 \end{pmatrix}$$

- the set $\{x, y, x^2\}$ is **closed**: the next value of each variable is a linear combination of the same three, so no new variable is needed
- forecasting becomes a matrix power, $\mathbf{z}_{k+n} = K^n \mathbf{z}_k$

When the Set Never Closes

Take the simplest nonlinear map, $x_{k+1} = x_k^2$, and try the same trick.

$$x \;\to\; x^2 \;\to\; x^4 \;\to\; x^8 \;\to\; \cdots$$

- to follow x linearly we need x^2, to follow x^2 we need x^4, and so on
- every new variable asks for another one, so no finite set closes
- a linear description still exists, but it needs infinitely many variables

Linear in many variables, or nonlinear in few, is the trade-off of this whole section.

Observables

An **observable** is any function of the state, $g(\mathbf{x})$, a number we could measure.

- a coordinate, $g(\mathbf{x}) = x$, or its square, $g(\mathbf{x}) = x^2$
- the energy of an oscillator, $g(x, v) = \frac{1}{2}v^2 + \frac{1}{2}\omega^2 x^2$
- the measured signal itself, $s_k = h(\mathbf{x}_k)$, the laser intensity, the convection speed

The lifting of the previous frames replaced the state (x, y) by three observables, x, y and x^2.

The Koopman Operator

For dynamics $\mathbf{x}_{k+1} = \mathbf{S}(\mathbf{x}_k)$ we ask how an observable changes in one step.

Definition (Koopman operator)

$$(\mathcal{K}g)(\mathbf{x}) \coloneqq g(\mathbf{S}(\mathbf{x}))$$

$\mathcal{K}g$ is the observable "g one step later", as a function of the present state

In the example, $\mathbf{S}(x, y) = (ax,\ by + cx^2)$:

$$\mathcal{K}x = a\,x, \qquad \mathcal{K}x^2 = a^2x^2, \qquad \mathcal{K}y = b\,y + c\,x^2$$

The matrix K of the lifting is $\mathcal{K}$ written in the basis (x, y, x^2).

B. O. Koopman, PNAS 17, 315 (1931), doi:10.1073/pnas.17.5.315

The Koopman Operator Is Linear

For any two observables and any numbers α, β,

$$\mathcal{K}(\alpha g_1 + \beta g_2)(\mathbf{x}) = \alpha\, g_1(\mathbf{S}(\mathbf{x})) + \beta\, g_2(\mathbf{S}(\mathbf{x})) = \alpha\, (\mathcal{K} g_1)(\mathbf{x}) + \beta\, (\mathcal{K} g_2)(\mathbf{x}).$$

- this holds for any dynamics $\mathbf{S}$, however nonlinear
- the price is the space of all observables, which is infinite-dimensional
- a finite matrix appears only when a finite set of observables closes

As an analogy, in the Heisenberg picture of quantum mechanics the observables evolve, linearly, and the state stays fixed.

Eigenfunctions

Some observables are only multiplied by a number at each step.

Definition (Koopman eigenfunction)

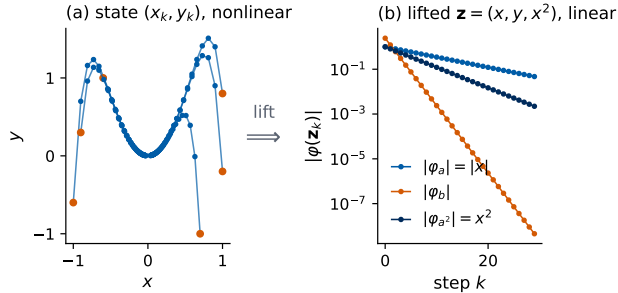
$$\mathcal{K}\varphi = \mu\,\varphi \quad \Longrightarrow \quad \varphi(\mathbf{x}_k) = \mu^k\,\varphi(\mathbf{x}_0)$$

μ: Koopman eigenvalue

- in the example $\varphi = x$ ($\mu = a$), $\varphi = x^2$ ($\mu = a^2$) and $\varphi = y + \frac{c}{b-a^2}x^2$ ($\mu = b$)
- for the oscillator $x \pm iv/\omega$ rotate with $|\mu| = 1$ at the frequency ω, and the energy has $\mu = 1$
- a sum of eigenfunctions is forecast exactly, $g(\mathbf{x}_k) = \sum_j c_j \mu_j^k \varphi_j(\mathbf{x}_0)$

S. L. Brunton, M. Budišić, E. Kaiser, J. N. Kutz, SIAM Rev. 64, 229 (2022), doi:10.1137/21M1401243

Eigenfunctions of the Example



Left trajectories in (x, y); right $|\varphi|$ of the three eigenfunctions along one trajectory, logarithmic axis.

- the trajectories curve, while each eigenfunction is a straight line, $|\varphi(\mathbf{x}_k)| = |\mu|^k |\varphi(\mathbf{x}_0)|$
- the matrix power K^n reproduces direct iteration to 4×10^{-16}, round-off

A Finite Dictionary Fitted from Data

In practice we do not know the closed set. We choose M observables, a **dictionary** $\boldsymbol{\psi} = (\psi_1, \ldots, \psi_M)$, and fit the best matrix from data.

$$K = \arg\min_K \sum_k \left\| \boldsymbol{\psi}(\mathbf{x}_{k+1}) - K\boldsymbol{\psi}(\mathbf{x}_k) \right\|^2$$

- one least-squares problem, as for AR
- with $\boldsymbol{\psi} = \mathbf{x}$ this is dynamic mode decomposition (DMD), with a richer dictionary extended DMD
- if the dictionary is not closed, a residual remains and the forecast drifts away

P. J. Schmid, J. Fluid Mech. 656, 5 (2010), doi:10.1017/S0022112010001217; M. O. Williams *et al.*, J. Nonlinear Sci. 25, 1307 (2015), doi:10.1007/s00332-015-9258-5

Delays as a Dictionary

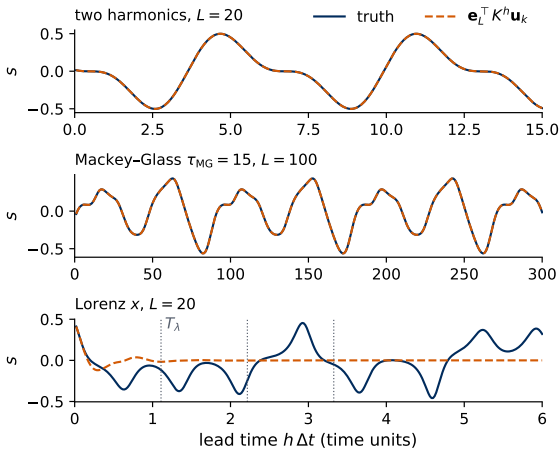
From a measured signal we know only one observable, $s_k = h(\mathbf{x}_k)$. Its past values are further observables, and Takens' theorem (Lecture 7) says they carry the state.

$$\psi_k = \mathbf{u}_k = \big(s_k, s_{k-1}, \ldots, s_{k-L+1}\big), \qquad \mathbf{u}_{k+1} \approx K\,\mathbf{u}_k$$

- all rows of K but one only shift the window, the remaining row predicts the new value
- that row is an AR(L) model, so AR is a Koopman approximation with the delay dictionary
- NVAR adds products of delays, a larger dictionary

F. Takens, Lect. Notes Math. 898, 366 (1981), doi:10.1007/BFb0091924

Periodic Signals Close, Chaos Does Not



Iterating $\mathbf{u}_{k+n} = K^n \mathbf{u}_k$ from one test step, against the truth.

- two sines and MG 15 continue for 300 steps: for periodic signals the delay dictionary closes
- Lorenz x relaxes to its mean within about one Lyapunov time
- chaos has no finite closed set, so a fixed finite dictionary fails at long horizons

From Fixed to Learned Features

- the Koopman operator explains why a linear readout on good features can forecast nonlinear dynamics
- AR uses the delays, NVAR the delays and their products, both chosen by hand
- for chaos no finite dictionary is exact, so the quality of the features decides the forecast
- the rest of this lecture lets a network produce the features, either a trained one (recurrent networks) or a fixed random one (reservoir)

3

Learned Features



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Features Produced by a Network

All our forecasts are a linear readout of features of the past,

$$\hat{\mathbf{y}}_k = W\,\varphi(\mathbf{v}_k) + \mathbf{b}.$$

- in AR and NVAR we choose the feature map φ ourselves, the delays and their products, and only W and $\mathbf{b}$ are fitted
- a neural network makes φ itself a function with adjustable parameters, which are fitted together with the readout
- we first build such a network from simple units and then give it a memory of its own

Artificial Neuron

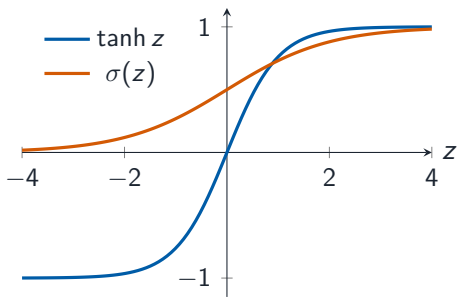
A neural network is assembled from many copies of one simple unit. The unit forms a weighted sum of its inputs and passes it through a fixed nonlinear function.

Definition (Unit with activation function ϕ)

$$a = \phi(\mathbf{w}^\top \mathbf{x} + b)$$

- $\mathbf{x}$ are the inputs, $\mathbf{w}$ the weights and b the bias of the unit
- $\mathbf{w}$ and b are trainable, while the activation function ϕ is chosen by us and stays fixed
- without ϕ, stacking units would only give another linear map

Two Activation Functions



$$\tanh z := \frac{e^z - e^{-z}}{e^z + e^{-z}}, \qquad \sigma(z) := \frac{1}{1 + e^{-z}}$$

- tanh is nearly linear for small $|z|$ and saturates at ± 1, where its slope $1 - \tanh^2 z$ goes to zero
- the logistic sigmoid σ takes values between 0 and 1 and will later act as a switch

Multilayer Perceptron on the Window

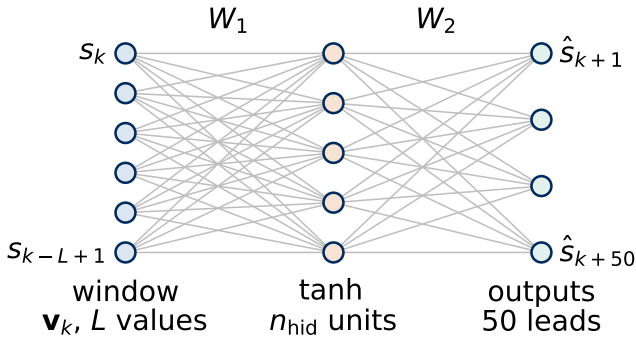
A layer of n_{hid} units reads the window and produces the features, and a linear readout maps them to the 50 leads.

Definition (Multilayer perceptron (MLP) forecaster)

$$\hat{\mathbf{y}}_k = W_2 \tanh\left(W_1 \mathbf{v}_k + \mathbf{b}_1\right) + \mathbf{b}_2$$

- **trainable parameters**: the hidden layer $W_1 \in \mathbb{R}^{n_{\mathrm{hid}} \times L}$, $\mathbf{b}_1$ and the readout $W_2 \in \mathbb{R}^{50 \times n_{\mathrm{hid}}}$, $\mathbf{b}_2$
- **hyperparameters**: the window length $L = 64$ and the number of hidden units $n_{\mathrm{hid}} = 64$, which give 7410 trainable parameters
- the structure is that of NVAR, with learned features in place of fixed products

MLP Architecture



Nodes are values and edges are weights. Every input feeds every hidden unit, and every hidden unit feeds every output.

- the network treats the window as a list of L numbers, with no built-in notion of which sample came first

Universal Approximation

Definition (Universal approximation theorem)

For every continuous f on a compact set $K \subset \mathbb{R}^L$ and every $\varepsilon > 0$ there are n_{hid}, W_1, $\mathbf{b}_1$, $\mathbf{w}_2$ with

$$\sup_{\mathbf{v} \in K} \left| f(\mathbf{v}) - \mathbf{w}_2^{\top} \tanh(W_1 \mathbf{v} + \mathbf{b}_1) \right| < \varepsilon.$$

- one hidden layer can thus represent any continuous forecast map of the window, but the theorem does not say how many units are needed or whether training finds the weights
- it concerns functions of a fixed window, so inputs older than L steps stay out of reach

G. Cybenko, Math. Control Signals Syst. 2, 303 (1989), doi:10.1007/BF02551274; K. Hornik, Neural Netw. 4, 251 (1991), doi:10.1016/0893-6080(91)90009-T

Loss and Gradient Descent

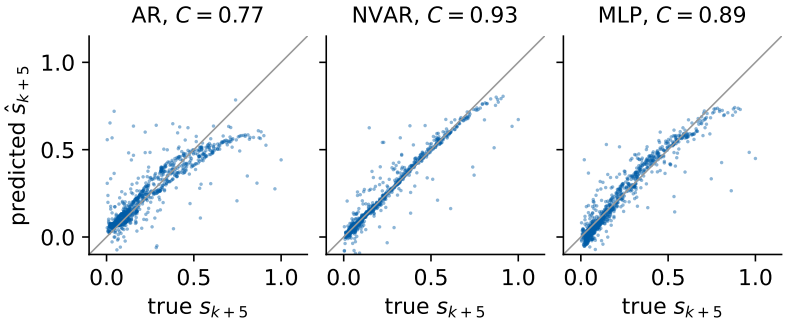
The weights of the hidden layer enter nonlinearly, so no single linear solve gives them. We collect all trainable parameters in $\boldsymbol{\theta}$ and lower the mean-square forecast error in many small steps.

$$\mathcal{L}(\boldsymbol{\theta}) \coloneqq \frac{1}{50\,B} \sum_{k \in \text{batch}} \left\| \hat{\mathbf{y}}_k(\boldsymbol{\theta}) - \mathbf{y}_k \right\|^2, \qquad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \nabla_{\boldsymbol{\theta}} \mathcal{L}$$

- each step uses a minibatch of B training windows, and the learning rate η sets the step size
- the gradient is computed by automatic differentiation
- the loss is non-convex in $\boldsymbol{\theta}$, so the result depends on the random initial weights

D. E. Rumelhart, G. E. Hinton, R. J. Williams, Nature 323, 533 (1986), doi:10.1038/323533a0

Learned Features on the Laser



Each panel plots the forecast $\hat{s}_{k+5}$ against the true value s_{k+5} over the Santa Fe test block, with $C = r^2$ in the title.

- AR compresses the large pulses, while NVAR and the MLP stay close to the diagonal
- the MLP levels off above true values of about 0.75

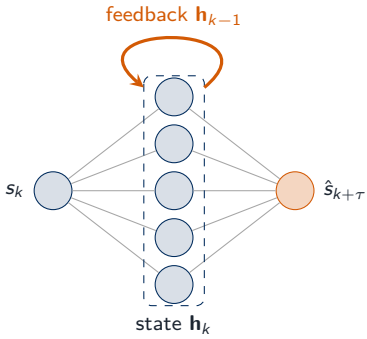
Memory Limited by the Window

The MLP, like AR and NVAR, is a function of the last L values and of nothing older.

- an input older than the window has no influence on the forecast, whatever the model
- a longer memory needs a longer window, and the number of parameters grows with it, linearly for the MLP and quadratically for NVAR
- the window length must be fixed in advance

A physical system stores its past differently. Its present state summarises the whole history of the forces that acted on it.

A State Carries the Past



We give the model a hidden state, updated by every new input and fed back at the next step,

$$\mathbf{h}_k = G(\mathbf{h}_{k-1}, s_k), \qquad \hat{\mathbf{y}}_k = F(\mathbf{h}_k)$$

- $\mathbf{h}_k$ depends on $s_k, s_{k-1}, s_{k-2}, \ldots$, and the number of parameters does not grow with the length of the history
- the update G is a dynamical system driven by the signal, and its stability decides how long the past is remembered

J. L. Elman, Cogn. Sci. 14, 179 (1990), doi:10.1207/s15516709cog1402_1

Four Ways to Process the Past

The models of Lectures 7 and 8 differ in where they store the past and in which part is trained.

	features fixed, readout trained	features and readout trained
window $\mathbf{v}_k$	AR, NVAR	MLP
state $\mathbf{h}_k$	reservoir	recurrent networks

- fixed features lead to one ridge regression, a convex problem with a unique solution
- trained features need gradient descent on a non-convex loss

4

Recurrent Neural Networks



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Recurrent Cell

The simplest trainable state update is one tanh layer that reads the old state together with the new input.

Definition (Recurrent neural network (RNN))

$$\mathbf{h}_k = \tanh\left(W_h \mathbf{h}_{k-1} + W_x s_k + \mathbf{b}\right), \qquad \hat{\mathbf{y}}_k = W_y \mathbf{h}_k + \mathbf{b}_y$$

- $W_h \in \mathbb{R}^{n_h \times n_h}$ couples the hidden units to each other, $W_x \in \mathbb{R}^{n_h}$ feeds in the signal, and W_y, $\mathbf{b}_y$ read out the 50 leads
- the state stays bounded, $\mathbf{h}_k \in (-1, 1)^{n_h}$, whatever the input

J. L. Elman, Cogn. Sci. 14, 179 (1990), doi:10.1207/s15516709cog1402_1

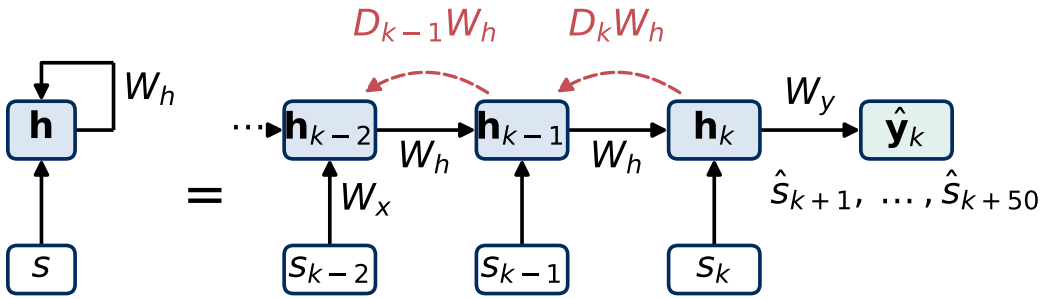
Trainable Parameters of the RNN

The same cell, with the same weights, is applied at every time step.

$$P_{\mathrm{RNN}} = \underbrace{n_h^2}_{W_h} + \underbrace{n_h}_{W_x} + \underbrace{n_h}_{\mathbf{b}} + \underbrace{50\,(n_h + 1)}_{W_y,\ \mathbf{b}_y}$$

- **trainable parameters**: W_h, W_x, $\mathbf{b}$ and the readout W_y, $\mathbf{b}_y$
- **hyperparameters**: the hidden size $n_h = 32$ and the window length $L = 64$, which give 2738 parameters, against 7410 for the MLP
- the number of parameters does not depend on how many steps the cell reads

Unrolling in Time



On the left the cell with its self-loop W_h, on the right one copy of the cell per time step. Dashed red arrows show the backward pass of training.

- unrolled, the RNN is a deep network with one layer per input value, and all layers share W_h, W_x and $\mathbf{b}$

Forecasting with a Recurrent Network

We use the recurrent network as a direct forecaster on the same windows as the MLP.

1. start from the state $\mathbf{h} = 0$ at the beginning of the window $\mathbf{v}_k$
2. feed the L values of the window one after the other through the cell
3. read the 50 leads from the last state, $\hat{\mathbf{y}}_k = W_y \mathbf{h}_k + \mathbf{b}_y$

The MLP and the recurrent networks thus see the same past values, so any difference in their forecasts comes from the way they process the window.

Backpropagation Through Time

The recurrent weights act at every step of the unrolled network, so their gradient collects one contribution from every step. The chain rule, run backwards from the last step, gives

$$\frac{\partial \mathcal{L}}{\partial W_h} = \sum_{j=1}^{k} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_j} \frac{\partial^+ \mathbf{h}_j}{\partial W_h}.$$

- this is backpropagation through time (BPTT), and $\partial^+ \mathbf{h}_j / \partial W_h$ is the direct effect of W_h at step j
- an input $k - j$ steps in the past reaches the loss only through the factor $\partial \mathbf{h}_k / \partial \mathbf{h}_j$

P. J. Werbos, Proc. IEEE 78, 1550 (1990), doi:10.1109/5.58337; Y. Bengio, P. Simard, P. Frasconi, IEEE Trans. Neural Netw. 5, 157 (1994), doi:10.1109/72.279181

Product of Step Jacobians

Each step of the cell contributes one Jacobian matrix to the path from $\mathbf{h}_j$ to $\mathbf{h}_k$.

$$\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_j} = D_k W_h \, D_{k-1} W_h \cdots D_{j+1} W_h, \qquad D_i \coloneqq \operatorname{diag}(1 - \mathbf{h}_i^2)$$

- D_i holds the slopes of tanh at step i, all between 0 and 1
- a contribution from m steps back is multiplied by m such factors
- the slopes depend on the trajectory, so the product depends on the input as well as on W_h

Vanishing and Exploding Gradients

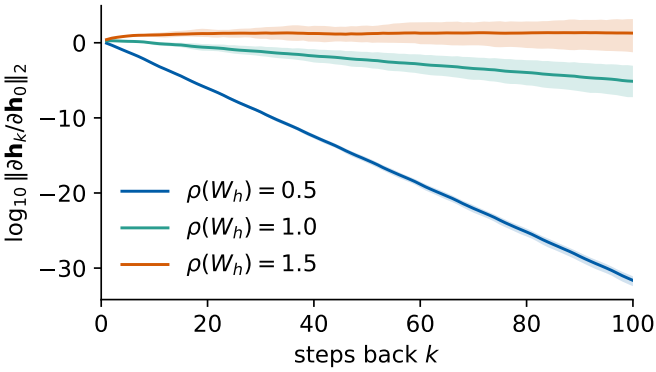
A product of m similar factors grows or decays roughly exponentially in m,

$$\left\| \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_j} \right\|_2 \approx e^{\gamma\,(k-j)}.$$

- for $\gamma < 0$ the gradient vanishes, and distant inputs drop out of the training signal
- for $\gamma > 0$ the gradient explodes, and rare very large updates spoil the weights
- learning dependencies over many steps requires γ close to 0

Y. Bengio, P. Simard, P. Frasconi, IEEE Trans. Neural Netw. 5, 157 (1994), doi:10.1109/72.279181;
R. Pascanu, T. Mikolov, Y. Bengio, ICML 2013, PMLR 28, 1310, arXiv:1211.5063

Gradient Norms in Random Networks



The plot shows $\log_{10} \|\partial \mathbf{h}_k / \partial \mathbf{h}_0\|_2$ against the step k for untrained RNNs driven by the Santa Fe signal. The spectral radius ρ is the largest eigenvalue modulus of W_h.

- the fitted rate is $\gamma = -0.736$ per step at $\rho = 0.5$ and $+0.003$ at $\rho = 1.5$
- at large ρ the tanh units saturate, their slopes shrink and stop the growth

Lyapunov Exponents of the Hidden Dynamics

The product of step Jacobians is the tangent map of Lecture 7, applied to the hidden state driven by the input. Its growth rate is therefore a finite-time Lyapunov exponent of the cell,

$$\lambda^{(m)} := \frac{1}{m} \ln \left\| D_{j+m} W_h \cdots D_{j+1} W_h \right\|_2 .$$

- vanishing gradients mean contracting hidden dynamics, exploding gradients mean expanding hidden dynamics
- this exponent belongs to the network and differs from the exponent λ_1 of the signal
- long memory needs the boundary between the two regimes, close to $\lambda^{(m)} = 0$

Remedies

Several standard measures keep the gradient under control.

measure	effect
gradient clipping R. Pascanu *et al.*, arXiv:1211.5063	rescales a gradient whose norm exceeds a threshold, against exploding gradients
orthogonal initial W_h A. M. Saxe *et al.*, arXiv:1312.6120	all singular values equal 1, so W_h alone neither shrinks nor stretches the product
gated cells Hochreiter, Schmidhuber (1997)	an additive memory path with a learned decay, against vanishing gradients
fixed recurrence (reservoir)	no gradient through time at all

Clipping cannot help against vanishing gradients. Gated cells were designed for that case.

5

Training Recurrent Networks



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Rescaling Avoids Saturation

Tanh units respond to inputs of order one. Larger inputs push them into saturation, where the slope of tanh is nearly zero and the gradient cannot pass. We therefore rescale the signal with the minimum and maximum of the training part,

$$s_k \;\to\; \frac{s_k - \min s}{\max s - \min s}.$$

- the raw Santa Fe intensity is recorded as integers up to 255, and in an untrained RNN fed with it 99.1% of the hidden values satisfy $|h| > 0.99$
- on the rescaled signal this fraction drops to 0.0%

Minibatches and the Adam Optimiser

Neighbouring training windows share all but one value, so we shuffle the windows before every pass and cut them into minibatches. One pass over all training windows is called an epoch.

- plain gradient descent moves every parameter with the same learning rate η
- the Adam optimiser divides each component of the gradient by a running estimate of its typical size, so every parameter moves by a step of order η
- this helps when the gradients of different weights differ by orders of magnitude

D. P. Kingma, J. Ba, ICLR 2015, arXiv:1412.6980

Gradient Clipping

A single very large gradient, the exploding case, can undo many epochs of training. Clipping keeps its direction and limits its length,

$$\mathbf{g} \;\rightarrow\; \mathbf{g}\,\min\left(1, \frac{c}{\|\mathbf{g}\|}\right),$$

where $\mathbf{g}$ is the gradient of all parameters together and c a threshold chosen by us.

- on Santa Fe the median gradient norm per epoch lay between 0.026 and 0.110, and no update was clipped
- here clipping is a safeguard against rare events that did not occur

R. Pascanu, T. Mikolov, Y. Bengio, ICML 2013, PMLR 28, 1310, arXiv:1211.5063

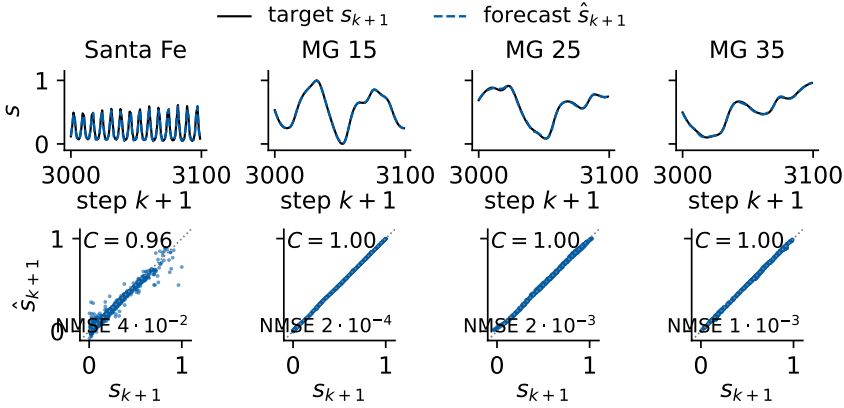
Early Stopping

A network with thousands of weights can keep lowering the training loss by fitting details of the training windows that do not repeat. The validation block detects this.

- after every epoch we compute the loss on the validation windows
- we keep the parameters of the epoch with the lowest validation loss and stop when it has not improved for a fixed number of epochs
- early stopping is our only protection against overfitting, and the test block is never used for it

L. Prechelt, in *Neural Networks: Tricks of the Trade*, Springer, 55 (1998), doi:10.1007/3-540-49430-8_3

One-Step RNN Forecasts



On the laser $C = 0.959$. On Mackey–Glass C is close to 1, but the error is larger than for AR in Lecture 7: the network fits all 50 leads at once by gradient descent, which does not reach the precision of a linear solve.

6

Gated Recurrent Networks



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Gates

The RNN rewrites its whole state at every step through a matrix and a tanh. A gate lets the network decide, unit by unit, how much of a vector to pass on.

Definition (Gate)

$$\mathbf{a}_k = \sigma\big(W_a[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_a\big), \qquad \mathbf{u} \;\to\; \mathbf{a}_k \odot \mathbf{u}$$

- $[\mathbf{h}_{k-1}, s_k]$ joins the old state and the new input into one vector, and $\odot$ multiplies component by component
- a gate value near 1 passes a component and a value near 0 blocks it
- the gate values depend on the current input and state, so the gates open and close in time

LSTM Cell State and Forget Gate

The long short-term memory (LSTM) network keeps a second state vector, the cell state $\mathbf{c}_k$, next to the hidden state $\mathbf{h}_k$. A forget gate decides how much of the old cell state is kept,

$$\mathbf{f}_k = \sigma\big(W_f[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_f\big), \qquad \mathbf{c}_k = \mathbf{f}_k \odot \mathbf{c}_{k-1} + \text{new content}.$$

- $\mathbf{f}_k = 1$ keeps a component unchanged and $\mathbf{f}_k = 0$ erases it
- the old cell state enters through an element-wise product only, with no matrix and no tanh

S. Hochreiter, J. Schmidhuber, Neural Comput. 9, 1735 (1997), doi:10.1162/neco.1997.9.8.1735; F. A. Gers, J. Schmidhuber, F. Cummins, Neural Comput. 12, 2451 (2000), doi:10.1162/089976600300015015

Input Gate and Candidate

New content is proposed by a tanh layer, the candidate $\mathbf{g}_k$, and admitted by a second gate, the input gate $\mathbf{i}_k$.

$$\mathbf{g}_k = \tanh\left(W_g[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_g\right)$$
$$\mathbf{i}_k = \sigma\left(W_i[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_i\right)$$
$$\mathbf{c}_k = \mathbf{f}_k \odot \mathbf{c}_{k-1} + \mathbf{i}_k \odot \mathbf{g}_k$$

- forgetting and writing are controlled separately, so a unit can keep its content and ignore the current input
- the cell state changes by addition, step after step

Output Gate and Hidden State

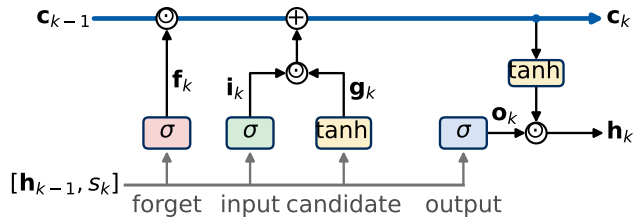
The cell state is read through a third gate, the output gate $\mathbf{o}_k$. The result is the hidden state, used by the next step and by the readout.

$$\mathbf{o}_k = \sigma(W_o[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_o), \qquad \mathbf{h}_k = \mathbf{o}_k \odot \tanh(\mathbf{c}_k)$$

- a unit can hold information in $\mathbf{c}_k$ without showing it in $\mathbf{h}_k$
- the cell state is unbounded, while the hidden state stays in $(-1, 1)$
- the readout to the 50 leads acts on $\mathbf{h}_k$, as in the RNN

LSTM Cell

$$
\begin{aligned}
\mathbf{f}_k &= \sigma(W_f[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_f) \\
\mathbf{i}_k &= \sigma(W_i[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_i) \\
\mathbf{g}_k &= \tanh(W_g[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_g) \\
\mathbf{o}_k &= \sigma(W_o[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_o) \\
\mathbf{c}_k &= \mathbf{f}_k \odot \mathbf{c}_{k-1} + \mathbf{i}_k \odot \mathbf{g}_k \\
\mathbf{h}_k &= \mathbf{o}_k \odot \tanh(\mathbf{c}_k)
\end{aligned}
$$



- the blue line on top carries the cell state, touched only by a product and a sum
- the four blocks read the same input $[\mathbf{h}_{k-1}, s_k]$, each with its own weights

Gradient Along the Cell State

Along the cell state alone, without the paths through the gates, the step Jacobian is diagonal,

$$\left.\frac{\partial \mathbf{c}_k}{\partial \mathbf{c}_{k-1}}\right|_{\text{cell path}} = \operatorname{diag}(\mathbf{f}_k), \qquad \left.\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}\right|_{\text{RNN}} = D_k W_h.$$

- no matrix W_h and no tanh slope enter this path, only the forget-gate values
- with $\mathbf{f}_k$ close to 1 the gradient flows back over many steps without decay
- Hochreiter and Schmidhuber called this path the constant error carousel

S. Hochreiter, J. Schmidhuber, Neural Comput. 9, 1735 (1997), doi:10.1162/neco.1997.9.8.1735

Memory Time of a Gated Unit

If the forget value of one unit stays at a constant f, its content decays exponentially, like a relaxing physical quantity,

$$f^m = e^{-m/\tau_{\mathrm{mem}}}, \qquad \tau_{\mathrm{mem}} := -\frac{1}{\ln f}.$$

- m counts the steps, and τ_{mem} is the memory time of the unit in steps
- for f close to 1, $\tau_{\mathrm{mem}} \approx 1/(1-f)$, so a long memory needs a forget value very close to 1
- the forget gate is a learnable relaxation time, and training sets it unit by unit

Memory Times Learned on the Laser

For every unit of the LSTM trained on Santa Fe we average the forget value f over all steps of the test windows and compute $\tau_{\mathrm{mem}} = -1/\ln \bar{f}$.

	smallest	median	largest
τ_{mem} over the 32 units (steps)	1.1	2.5	3.5

- all memory times are a few steps, much shorter than the window of $L = 64$ values

Trainable Parameters of the LSTM

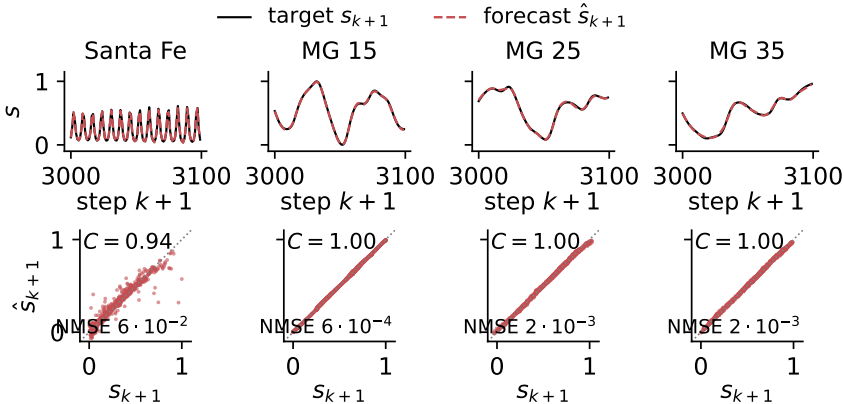
Each of the four blocks, three gates and the candidate, has its own matrix acting on $[\mathbf{h}_{k-1}, s_k]$ and its own bias.

$$P_{\mathrm{LSTM}} = 4\,(n_h^2 + n_h + n_h) + 50\,(n_h + 1)$$

- **trainable parameters**: W_f, W_i, W_g, W_o, the biases $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_g, \mathbf{b}_o$ and the readout W_y, $\mathbf{b}_y$
- **hyperparameters**: $n_h = 32$ and $L = 64$ as for the RNN, which give 6002 parameters, against 2738 for the RNN
- a lighter gated cell, the gated recurrent unit (GRU), is implemented in Lab 036

K. Cho *et al.*, EMNLP 2014, 1724, doi:10.3115/v1/D14-1179

One-Step LSTM Forecasts



On the laser $C = 0.937$, on Mackey–Glass close to 1, as for the RNN.

7

Reservoir Computing



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Reservoir Computing

Training the recurrent weights is the expensive and fragile part of a recurrent network. Reservoir computing leaves them untrained. A fixed dynamical system, the reservoir, is driven by the signal, and only a linear readout of its state is fitted,

$$\mathbf{x}_k = G(\mathbf{x}_{k-1}, s_k), \qquad \hat{\mathbf{y}}_k = W_{\text{out}}\,\mathbf{x}_k + \mathbf{b}.$$

- a good reservoir responds nonlinearly to its input, has many readable variables $\mathbf{x}_k \in \mathbb{R}^N$, and forgets its distant past
- as an analogy, the reservoir behaves like a driven, dissipative physical system whose state reflects the recent forcing

H. Jaeger, GMD Report 148 (2001); W. Maass, T. Natschläger, H. Markram, Neural Comput. 14, 2531 (2002), doi:10.1162/089976602760407955

Echo State Network

The echo state network (ESN) uses a random recurrent network as the reservoir.

Definition (Leaky echo state network)

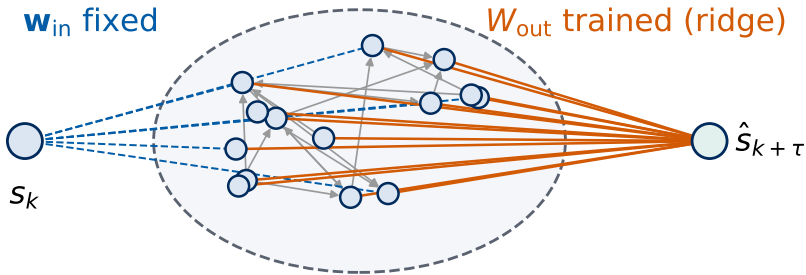
$$\mathbf{x}_k = (1-\alpha)\,\mathbf{x}_{k-1} + \alpha \tanh\left(W\mathbf{x}_{k-1} + \mathbf{w}_{\text{in}} s_k\right), \qquad \hat{\mathbf{y}}_k = W_{\text{out}}\,\mathbf{x}_k + \mathbf{b}$$

- $W \in \mathbb{R}^{N\times N}$ couples the reservoir units and $\mathbf{w}_{\text{in}} \in \mathbb{R}^N$ feeds in the signal, both drawn at random once
- the leak rate $\alpha \in (0, 1]$ slows the state down, and with $\alpha = 1$ the update is the RNN cell with random weights

H. Jaeger, H. Haas, Science 304, 78 (2004), doi:10.1126/science.1091277;

M. Lukoševičius, H. Jaeger, Comput. Sci. Rev. 3, 127 (2009), doi:10.1016/j.cosrev.2009.03.005

ESN Architecture



reservoir $\mathbf{x}_k$, W fixed (spectral radius ρ)

Nodes are reservoir units. Grey arrows are the sparse couplings W, dashed blue arrows the input weights $\mathbf{w}_{\text{in}}$ and orange arrows the readout.

- only the orange weights are fitted, and the drawing shows 14 units of the $N = 300$ used for forecasting

Building the Reservoir

The individual weights are random, and a few global numbers set the behaviour of the reservoir. We draw a sparse random matrix W_0 and rescale it,

$$W = \rho \frac{W_0}{\max_i |\lambda_i(W_0)|}, \qquad (\mathbf{w}_{\text{in}})_a \text{ uniform in } [-\sigma_{\text{in}}, \sigma_{\text{in}}].$$

- the spectral radius ρ, the largest eigenvalue modulus of W, sets how slowly perturbations of the state decay and with it the memory
- the input scaling σ_{in} sets how far the units are driven into the nonlinear range of tanh

Trainable and Fixed Parts of the ESN

The ESN separates three kinds of quantities.

- **trainable parameters**: only the readout W_{out} and $\mathbf{b}$
- **fixed random weights**: the couplings W and the input weights $\mathbf{w}_{\mathrm{in}}$, drawn once and never changed
- **hyperparameters**: the size N, the spectral radius ρ, the input scaling σ_{in}, the leak rate α and the ridge strength β of the readout

The hyperparameters are chosen on the validation block, as for AR and NVAR.

Training the Readout

The reservoir is run once over the whole signal from $\mathbf{x}_0 = 0$. Its states at the fitting origins form the rows of a feature matrix X and the targets the rows of Y, both with their means subtracted.

$$W_{\text{out}}^{\top} = (X^{\top}X + \beta I)^{-1} X^{\top} Y$$

- this is the ridge regression of AR and NVAR, with reservoir states in place of delays and products
- the first states still remember the arbitrary start $\mathbf{x}_0$, so this initial stretch, the washout, is left out
- fitting is one linear solve, with no gradient descent and no backpropagation through time

A. E. Hoerl, R. W. Kennard, Technometrics 12, 55 (1970), doi:10.1080/00401706.1970.10488634

Echo State Property

A readout of $\mathbf{x}_k$ can only be a function of the input if the state forgets where it started.

Definition (Echo state property)

For every input sequence and any two initial states, the driven states approach each other,

$$\|\mathbf{x}_k - \mathbf{x}'_k\| \to 0 \quad (k \to \infty),$$

so that $\mathbf{x}_k$ becomes a function of the input history $s_k, s_{k-1}, \ldots$ alone.

- the state is then an echo of the input, a fading memory of the drive
- in the language of dynamical systems the reservoir synchronises with its drive, called generalised synchronisation

H. Jaeger, GMD Report 148 (2001); N. F. Rulkov *et al.*, Phys. Rev. E 51, 980 (1995), doi:10.1103/PhysRevE.51.980;

M. Płodzień, *Lecture Notes on Time Series Forecasting. A Dynamical-Systems Approach*, ch. 6

Contraction and the Spectral Radius

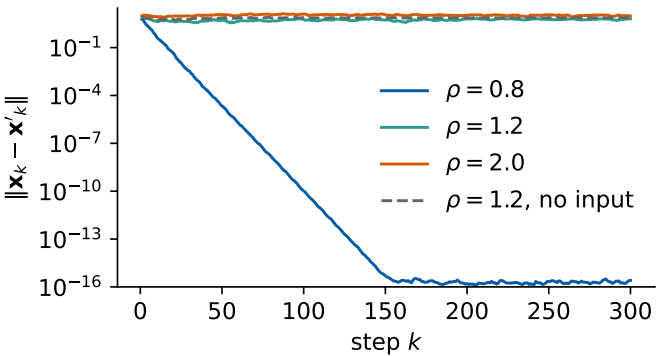
Two simple conditions on the couplings bracket the echo state property, here for $\alpha = 1$.

- tanh never stretches distances, $|\tanh a - \tanh b| \leq |a - b|$, so if the largest singular value of W is below 1 the update is a contraction and the property holds for every input
- if the spectral radius ρ exceeds 1, the zero state is unstable and the property fails at zero input
- between the two, with $\rho < 1$ but the largest singular value above 1, nothing is guaranteed

In practice ρ is used as the control parameter, and the property is checked directly.

I. B. Yildiz, H. Jaeger, S. J. Kiebel, Neural Netw. 35, 1 (2012), doi:10.1016/j.neunet.2012.07.005; M. Płodzień, *Lecture Notes on Time Series Forecasting. A Dynamical-Systems Approach*, ch. 6

Echo State Property Tested Directly



The plot shows the distance $\|\mathbf{x}_k - \mathbf{x}'_k\|$ against the step k for two copies of one reservoir with $N = 100$, started from different states and driven by the same Santa Fe input.

- at $\rho = 0.8$ the distance decays to 2.6×10^{-16}, round-off level
- at $\rho = 1.2$ and 2.0 it stays of order one, also without input (dashed), so the property fails

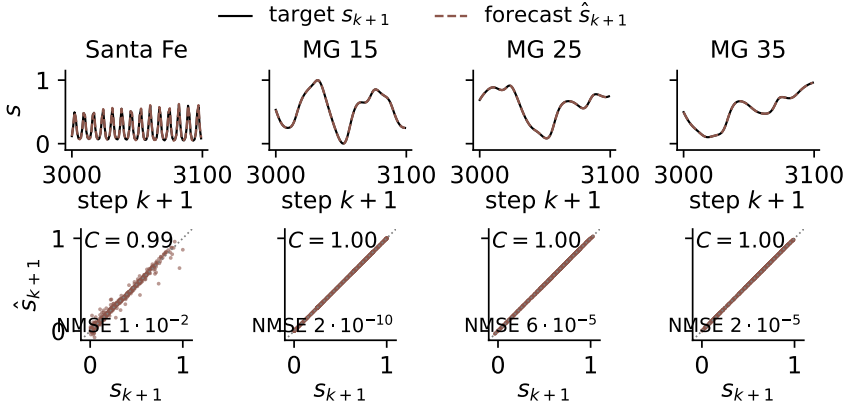
ESN Settings for Forecasting

For forecasting we use one random reservoir with $N = 300$ units for all signals. The validation block selects ρ, σ_{in} and α for each signal.

	Lorenz	MG 15	MG 25	MG 35	Santa Fe
spectral radius ρ	1.1	1.1	0.95	0.8	0.95
input scaling σ_{in}	1.0	0.5	1.0	1.0	0.1
leak rate α	1.0	1.0	0.3	0.3	1.0

- the slowly varying Mackey–Glass signals at $\tau_{\text{MG}} = 25$ and 35 select a leak rate below 1
- the selection does not test the echo state property, so for Lorenz and MG 15, with $\rho > 1$, it remains unchecked

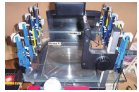
One-Step ESN Forecasts



On the laser $C = 0.987$, the best one-step value of all models, and on Mackey–Glass the readout is almost exact.

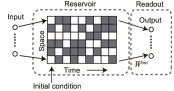
Physical Reservoirs

Nothing inside a reservoir is trained, so any physical system with a nonlinear response, fading memory and many readable quantities can serve.



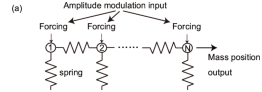
water in a bucket: the input drives waves, a camera image of the surface gives the features

C. Fernando, S. Sojakka, ECAL 2003, 588



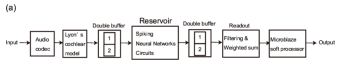
cellular automaton: input written into cells, a fixed local rule evolves them, the pattern is read out

O. Yilmaz, arXiv:1410.0162 (2014)



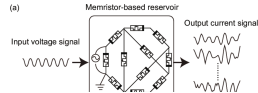
coupled oscillators: masses on springs driven by the input, positions read out

H. Hauser *et al.*, Biol. Cybern. 105, 355 (2011)



electronic circuit: one nonlinear node with a delay line, delayed samples act as virtual nodes

L. Appeltant *et al.*, Nat. Commun. 2, 468 (2011)

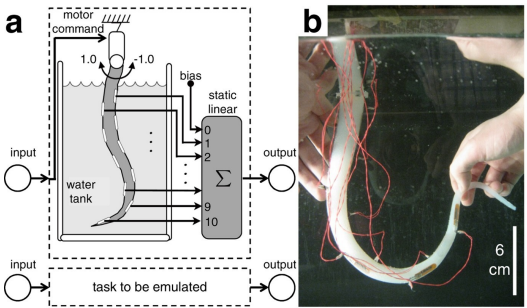


memristor network: resistance depends on the past current, so the network remembers

C. Du *et al.*, Nat. Commun. 8, 2204 (2017)

G. Tanaka *et al.*, Neural Netw. 115, 100 (2019), doi:10.1016/j.neunet.2019.03.005

Soft Silicone Arm as a Reservoir



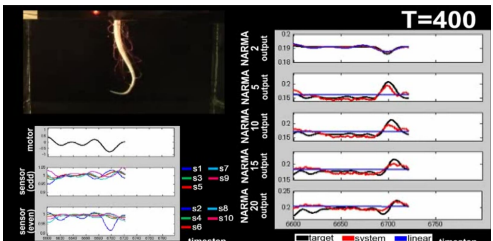
(a) motor command in, ten bend sensors out, weighted sum; (b) the arm under water

K. Nakajima, H. Hauser, T. Li, R. Pfeifer, Sci. Rep. 5, 10487 (2015), doi:10.1038/srep10487

A soft silicone arm, about 45 cm long, moves under water.

- the input $m(k) \in [-1, 1]$ is the rotation angle of the base
- the state is formed by ten bend sensors $x_i(k)$ along the arm
- water drag supplies the nonlinearity, and damping the fading memory

The Soft Arm in Motion



▶ video of the moving arm (course website)

original: Supplementary Movie S1 of Nakajima *et al.*
K. Nakajima, H. Hauser, T. Li, R. Pfeifer, Sci. Rep. 5, 10487 (2015), doi:10.1038/srep10487

The readout $\hat{y}_k = \sum_i w_i x_i(k) + w_0$ is a weighted sum of the ten sensor signals.

- only the weights w_i are fitted, by one linear solve, exactly as for the echo state network
- the body itself, with its drag and damping, plays the role of the random reservoir

8

Static Models and Recurrent Networks



JAGIELLONIAN UNIVERSITY
IN KRAKOW

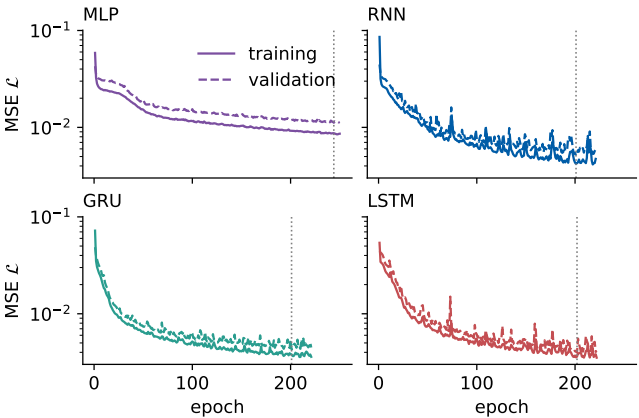
How the Models Are Compared

We compare the static models of Lecture 7, which read a fixed window of the past, with the two recurrent networks of this lecture. All forecast the 50 leads from the same rescaled signals and are scored on the same test steps.

model	reads	trained	fitting
persistence	last value s_k	nothing	none
AR, NVAR	up to 64 past values	readout	linear solve
MA	last 8 prediction errors	9 weights	likelihood
RNN, LSTM	$L = 64$ past values through a state	all weights	BPTT

The signals are the Santa Fe laser and Mackey–Glass at $\tau_{\mathrm{MG}} = 15$ (limit cycle), 25 (weak chaos) and 35 (developed chaos).

Training Curves on the Laser



The plot shows the mean-square error (MSE) loss against the training epoch on Santa Fe, solid for training and dashed for validation. The dotted line marks the epoch with the lowest validation loss.

- the recurrent networks reach lower losses than the MLP
- training and validation curves stay close at these network sizes

Dependence on the Random Seed

The random seed fixes the initial weights and the order of the minibatches. The table gives the test C_{tot} on Santa Fe for three seeds.

	seed 0	seed 1	seed 2	mean
MLP	35.3	35.6	36.0	35.6
RNN	42.5	41.6	42.2	42.1
LSTM	44.3	44.0	44.9	44.4

- the step from the MLP to any recurrent network is several times larger than the spread over seeds
- on the other signals the networks were trained with one seed only, so their seed spread there is unknown

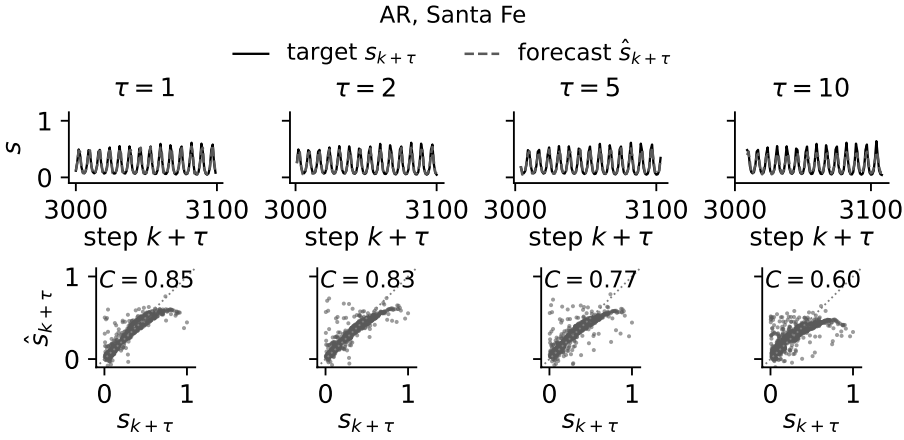
Forecasting the Laser at Several Leads

The capacity $C(\tau)$ summarises each lead by one number. Before comparing curves, we look at the forecasts themselves.

- each of the next frames shows one model on the Santa Fe laser at the leads $\tau = 1, 2, 5, 10$
- upper panels: target $s_{k+\tau}$ (black) and forecast $\hat{s}_{k+\tau}$ (dashed) on the first 100 test steps
- lower panels: forecast against target on all test steps, with $C(\tau)$

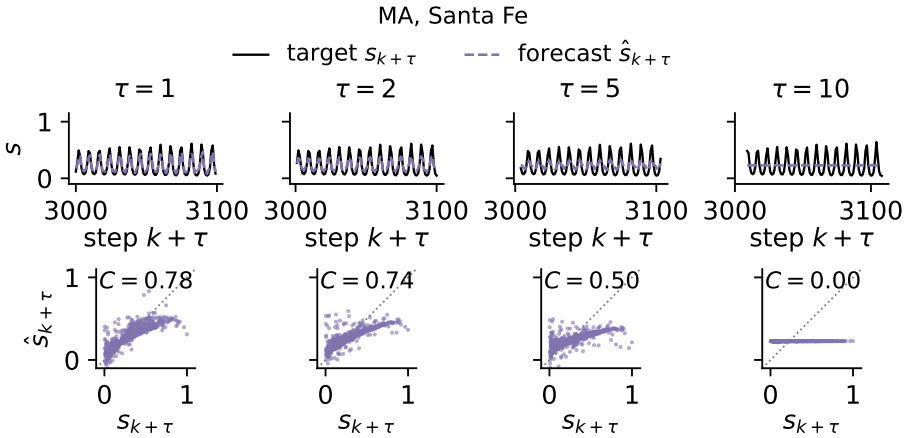
Watch how the points leave the diagonal as the lead grows.

AR Forecasts of the Laser at Several Leads



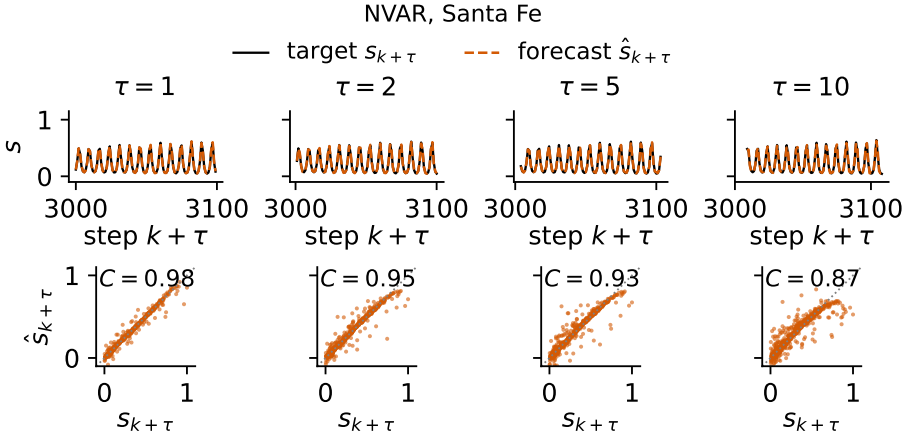
AR follows the flashes one step ahead ($C = 0.85$), underestimates the large ones and loses them by $\tau = 10$ ($C = 0.60$).

MA Forecasts of the Laser at Several Leads



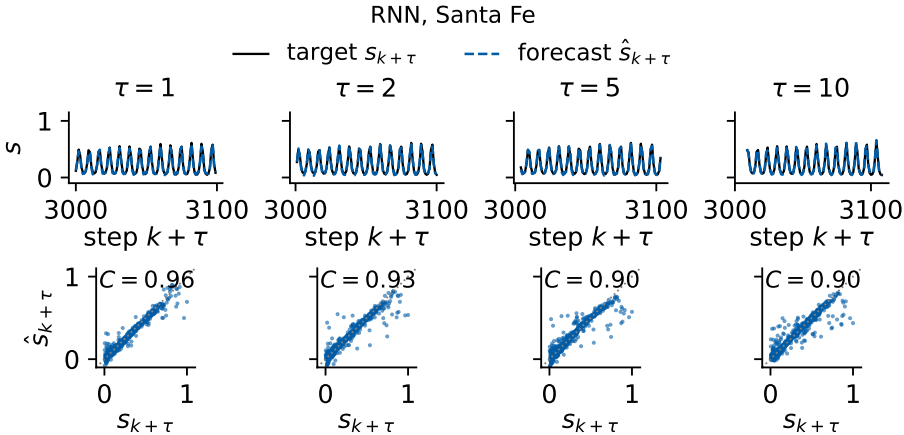
MA(8) uses the last 8 prediction errors, so beyond 8 steps it can only forecast the mean: $C = 0.50$ at $\tau = 5$ and 0.00 at $\tau = 10$.

NVAR Forecasts of the Laser at Several Leads



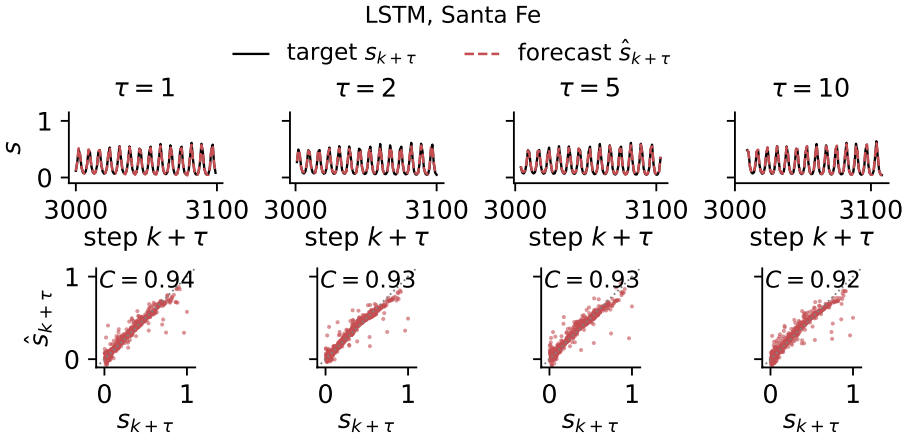
The products of past values follow the large flashes: $C = 0.98$ at one step and 0.87 at $\tau = 10$.

RNN Forecasts of the Laser at Several Leads



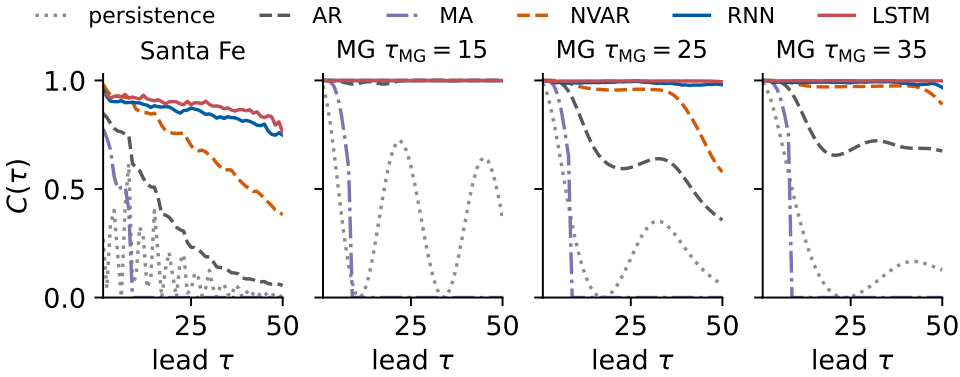
The recurrent network keeps $C = 0.90$ at $\tau = 10$, close to its one-step value 0.96.

LSTM Forecasts of the Laser at Several Leads



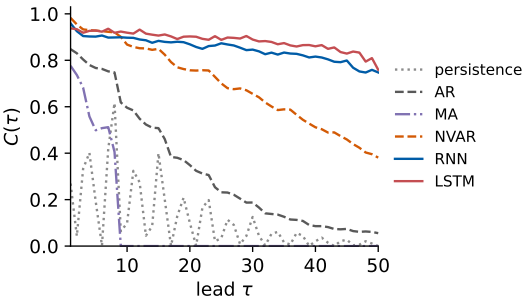
The LSTM degrades least with the lead, from $C = 0.94$ at one step to 0.92 at $\tau = 10$.

Capacity of Static and Recurrent Models



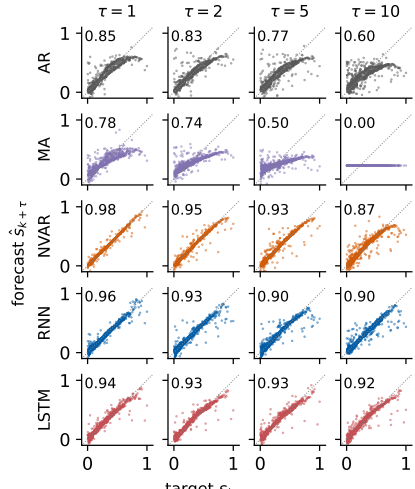
Capacity $C(\tau)$ on the test block against the lead $\tau = 1, \ldots, 50$, one panel per signal. We go through the signals one by one.

Linear and Nonlinear Models on the Laser



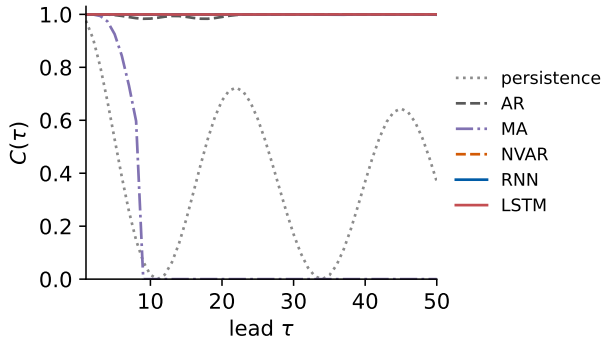
$C(\tau)$ against the lead τ.

- RNN and LSTM keep C high over all 50 leads, $C_{\text{tot}} = 42.5$ and 44.3
- NVAR reaches 34.7, AR 16.5, MA drops to zero after 8 steps



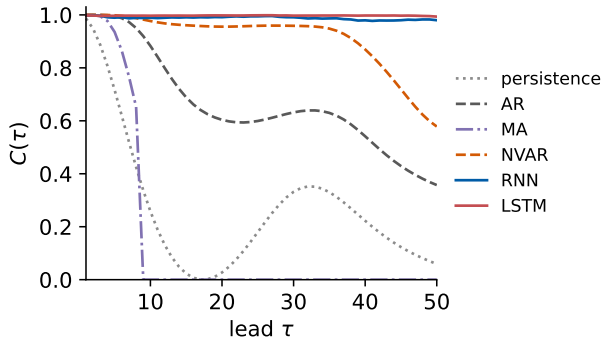
forecast against target, with $C(\tau)$

Mackey–Glass Limit Cycle



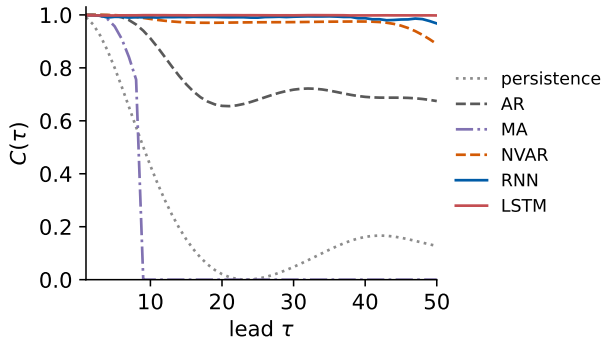
- a periodic signal is easy: AR, NVAR and both networks stay near $C = 1$ at all leads
- persistence oscillates with the period, and MA forgets after its 8 steps

Mackey–Glass in Weak Chaos



- AR decays to $C_{\mathrm{tot}} = 33.1$, NVAR holds longer ($C_{\mathrm{tot}} = 45.5$) and drops at large leads
- the RNN and the LSTM stay near 1 over all leads, $C_{\mathrm{tot}} = 49.4$ and 49.9

Mackey–Glass in Developed Chaos



- the picture is the same as for weak chaos, AR $C_{\text{tot}} = 38.0$, NVAR 48.6, LSTM 49.9
- all 50 leads lie within one Lyapunov time here, so the limit comes from how well a model learns the dynamics

Total Capacity

$C_{\mathrm{tot}} = \sum_{\tau=1}^{50} C(\tau)$ on the test block, out of a maximum of 50.

model	Santa Fe	MG 15	MG 25	MG 35
persistence	5.5	19.5	13.3	11.8
AR	16.5	49.8	33.1	38.0
MA	4.7	7.1	7.2	7.4
NVAR	34.7	50.0	45.5	48.6
RNN	42.5	50.0	49.4	49.5
LSTM	44.3	50.0	49.9	49.9

- a state that carries the past gives the largest gain on the laser
- the echo state network of the reservoir section reaches $C_{\mathrm{tot}} = 34.7$ on Santa Fe with a trained readout only

Teacher Forcing and Free Run

All forecasts so far read the measured past. A model that generates a signal on its own must instead feed back its own output. In a free run we iterate the one-step forecast F_1 from the last measured sample k_0,

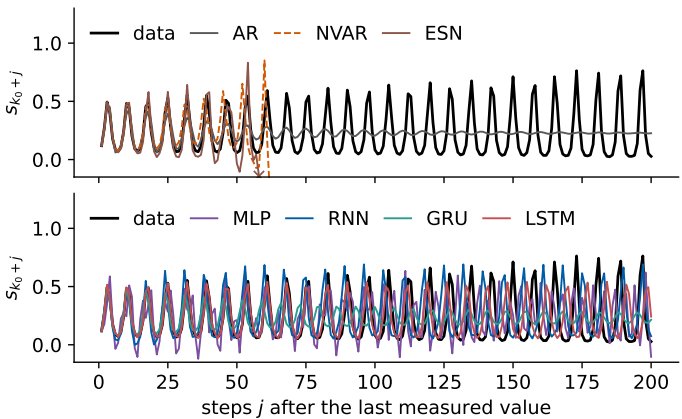
$$\hat{s}_{k_0+j} = F_1\big(\hat{s}_{k_0+j-1}, \ldots, \hat{s}_{k_0+j-L}\big), \qquad j = 1, 2, \ldots$$

where the window holds measured values for all times up to k_0.

- training always feeds measured values, which is called teacher forcing
- in a free run the model meets its own errors, which it never saw during training
- the iterated model is an autonomous dynamical system with an attractor of its own

R. J. Williams, D. Zipser, Neural Comput. 1, 270 (1989), doi:10.1162/neco.1989.1.2.270

Free Run on the Laser



Free run $\hat{s}_{k_0+j}$ against the step j after the last measured value, data in black. Arrows mark runs that leave the plotted range.

- AR relaxes to a small oscillation, NVAR and the ESN follow the pulses and then diverge
- the LSTM stays close to the data longest, then pulsates with a drifting phase

Free-Run Errors

Santa Fe free run from one origin and one seed. $j_{0.2}$ is the number of steps before the error $|\hat{s}_{k_0+j} - s_{k_0+j}|$ first exceeds 0.2, and the NMSE is the mean-square error divided by the variance of the data.

	AR	NVAR	ESN	RNN	LSTM
$j_{0.2}$	38	36	47	23	110
NMSE, 50 steps	0.24	0.43	0.15	0.56	0.01

- an NMSE near 1 or above means the phase is lost
- a single origin and seed make this ranking much less reliable than that of the direct forecasts

Cost of Fitting

The table gives the median wall-clock time of one fit over the signals, for the selected setting only, on one processor.

	AR	NVAR	ESN	RNN	LSTM
fit time	4 ms	159 ms	67 ms	18 s	46 s

- a ridge readout solves one linear system, while BPTT runs every window forward and backward through all L steps for hundreds of epochs
- the recurrent networks cost about a hundred (RNN) to several hundred (LSTM) times the ESN fit, and the extra cost pays off on the laser

9
Summary



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Models of Lectures 7 and 8

model	memory	features	trained
AR	window of past values	the values themselves	readout
MA	past prediction errors	the errors themselves	weights by likelihood
NVAR	window of past values	values and their products	readout
MLP	window of past values	learned tanh features	all weights
ESN	state of a random reservoir	fixed random tanh units	readout
RNN	learned state	learned	all weights
LSTM	learned gated state	learned	all weights

A readout is fitted by one linear solve, while all weights need gradient descent. Each row down the table adds flexibility and cost, and the result depends more on training choices.

Choosing a Forecasting Model

We characterise the signal first, by its spectrum, embedding, largest Lyapunov exponent λ_1 and Kaplan–Yorke dimension D_{KY}. The characterisation suggests where to start.

- for a periodic or nearly linear signal AR is hard to beat, and it fits in milliseconds
- for low-dimensional chaos with a smooth nonlinearity NVAR or an ESN, fitted by one linear solve, is a good first choice
- for folding dynamics, as in the laser, recurrent networks did best, at the price of gradient training and dependence on the seed

Checks Before Ranking Models

A ranking of models is only as reliable as the checks behind it.

- a reservoir should be tested for the echo state property, for example with two copies started from different states
- trained networks should be compared over several seeds, since the seed spread sets the smallest meaningful difference
- a good direct forecast says little about a free run, and no model forecasts chaos far beyond a few Lyapunov times

10
References



JAGIELLONIAN UNIVERSITY
IN KRAKOW

References 1/3

- L. Appeltant *et al.*, Nat. Commun. 2, 468 (2011), doi:10.1038/ncomms1476
- Y. Bengio, P. Simard, P. Frasconi, IEEE Trans. Neural Netw. 5, 157 (1994), doi:10.1109/72.279181
- S. L. Brunton, M. Budišić, E. Kaiser, J. N. Kutz, SIAM Rev. 64, 229 (2022), doi:10.1137/21M1401243
- K. Cho *et al.*, Proc. EMNLP 2014, 1724 (2014), doi:10.3115/v1/D14-1179
- G. Cybenko, Math. Control Signals Syst. 2, 303 (1989), doi:10.1007/BF02551274
- J. Dambre, D. Verstraeten, B. Schrauwen, S. Massar, Sci. Rep. 2, 514 (2012), doi:10.1038/srep00514
- C. Du *et al.*, Nat. Commun. 8, 2204 (2017), doi:10.1038/s41467-017-02337-y
- J. L. Elman, Cogn. Sci. 14, 179 (1990), doi:10.1207/s15516709cog1402_1
- C. Fernando, S. Sojakka, *Advances in Artificial Life* (ECAL 2003), Springer, 588 (2003), doi:10.1007/978-3-540-39432-7_63
- F. A. Gers, J. Schmidhuber, F. Cummins, Neural Comput. 12, 2451 (2000), doi:10.1162/089976600300015015
- H. Hauser, A. J. Ijspeert, R. M. Füchslin, R. Pfeifer, W. Maass, Biol. Cybern. 105, 355 (2011), doi:10.1007/s00422-012-0471-0
- S. Hochreiter, J. Schmidhuber, Neural Comput. 9, 1735 (1997), doi:10.1162/neco.1997.9.8.1735
- A. E. Hoerl, R. W. Kennard, Technometrics 12, 55 (1970), doi:10.1080/00401706.1970.10488634

References 2/3

- K. Hornik, Neural Netw. 4, 251 (1991), doi:10.1016/0893-6080(91)90009-T
- U. Hübner, N. B. Abraham, C. O. Weiss, Phys. Rev. A 40, 6354 (1989), doi:10.1103/PhysRevA.40.6354
- H. Jaeger, *The "echo state" approach to analysing and training recurrent neural networks*, GMD Report 148 (2001), ai.rug.nl/minds/uploads/EchoStatesTechRep.pdf
- H. Jaeger, H. Haas, Science 304, 78 (2004), doi:10.1126/science.1091277
- D. P. Kingma, J. Ba, ICLR 2015, arXiv:1412.6980
- B. O. Koopman, PNAS 17, 315 (1931), doi:10.1073/pnas.17.5.315
- E. N. Lorenz, J. Atmos. Sci. 20, 130 (1963), doi:10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2
- M. Lukoševičius, H. Jaeger, Comput. Sci. Rev. 3, 127 (2009), doi:10.1016/j.cosrev.2009.03.005
- W. Maass, T. Natschläger, H. Markram, Neural Comput. 14, 2531 (2002), doi:10.1162/089976602760407955
- M. C. Mackey, L. Glass, Science 197, 287 (1977), doi:10.1126/science.267326
- K. Nakajima, H. Hauser, T. Li, R. Pfeifer, Sci. Rep. 5, 10487 (2015), doi:10.1038/srep10487
- R. Pascanu, T. Mikolov, Y. Bengio, ICML 2013, PMLR 28, 1310 (2013), arXiv:1211.5063
- M. Płodzień, *Lecture Notes on Time Series Forecasting. A Dynamical-Systems Approach*, github.com/MarcinPlodzien/time-series-forecasting-book

References 3/3

- L. Prechelt, in *Neural Networks: Tricks of the Trade*, Springer, 55 (1998), doi:10.1007/3-540-49430-8_3
- N. F. Rulkov, M. M. Sushchik, L. S. Tsimring, H. D. I. Abarbanel, Phys. Rev. E 51, 980 (1995), doi:10.1103/PhysRevE.51.980
- D. E. Rumelhart, G. E. Hinton, R. J. Williams, Nature 323, 533 (1986), doi:10.1038/323533a0
- A. M. Saxe, J. L. McClelland, S. Ganguli, arXiv:1312.6120 (2013)
- P. J. Schmid, J. Fluid Mech. 656, 5 (2010), doi:10.1017/S0022112010001217
- F. Takens, Lect. Notes Math. 898, 366 (1981), doi:10.1007/BFb0091924
- G. Tanaka *et al.*, Neural Netw. 115, 100 (2019), doi:10.1016/j.neunet.2019.03.005
- P. J. Werbos, Proc. IEEE 78, 1550 (1990), doi:10.1109/5.58337
- M. O. Williams, I. G. Kevrekidis, C. W. Rowley, J. Nonlinear Sci. 25, 1307 (2015), doi:10.1007/s00332-015-9258-5
- R. J. Williams, D. Zipser, Neural Comput. 1, 270 (1989), doi:10.1162/neco.1989.1.2.270
- I. B. Yildiz, H. Jaeger, S. J. Kiebel, Neural Netw. 35, 1 (2012), doi:10.1016/j.neunet.2012.07.005
- O. Yilmaz, arXiv:1410.0162 (2014)